PLACEMENT CODING REVISION SHEET

Top DSA Patterns in Java

#1 Reverse Array

EASY TWO POINTERS *Swap from both ends.*

Time / Space	O(N) / O(1)	I/O Sample	In: [1,2,3] → Out: [3,2,1]

💡 **Interview Trick:** In-place modification avoids extra space overhead.

```java
static void reverse(int[] a) {
    int l = 0, r = a.length - 1;
    while (l < r) {
        int t = a[l]; a[l] = a[r]; a[r] = t;
        l++; r--;
    }
}
```

#2 Min and Max in Array

EASY LINEAR SCAN *Track both in one pass.*

Time / Space	O(N) / O(1)	I/O Sample	In: [3,1,5,2] → Out: [1,5]

💡 **Interview Trick:** Fewer comparisons than sorting the array.

```java
static int[] minMax(int[] a) {
    int mn = a[0], mx = a[0];
    for (int x : a) {
        if (x < mn) mn = x;
        if (x > mx) mx = x;
    }
    return new int[]{mn, mx};
}
```

#3 Count Vowels

EASY STRING/HASHING *Use 'aeiou'.indexOf(ch).*

Time / Space	O(N) / O(1)	I/O Sample	In: "hello" → Out: 2

💡 **Interview Trick:** Convert to lowercase first to simplify checks.

```
static int countVowels(String s) {
    int c = 0;
    for (char ch : s.toLowerCase().toCharArray())
        if ("aeiou".indexOf(ch) != -1) c++;
    return c;
}
```

#4 Palindrome Check

EASY TWO POINTERS *Compare left and right.*

Time / Space	O(N) / O(1)	I/O Sample	In: "racecar" → Out: true

💡 **Interview Trick:** Works efficiently without reversing the string.

```
static boolean isPal(String s) {
    int l = 0, r = s.length() - 1;
    while (l < r)
        if (s.charAt(l++) != s.charAt(r--)) return false;
    return true;
}
```

#5 Remove Duplicates

EASY TWO POINTERS *Use write index j.*

Time / Space	O(N) / O(1)	I/O Sample	In: [1,1,2] → Out: 2 (array: [1,2,...])

💡 **Interview Trick:** ⚠️ Works only for sorted arrays. Array must be sorted for this to work in O(1) space.

```
static int removeDup(int[] a) {
    if (a.length == 0) return 0;
    int j = 1;
    for (int i = 1; i < a.length; i++)
        if (a[i] != a[i - 1]) a[j++] = a[i];
    return j;
}
```

#6 Two Sum

EASY HASHING *HashMap for seen values.*

Time / Space	O(N) / O(N)	I/O Sample	In: [2,7,11,15], target=9 → Out: [0,1]

💡 **Interview Trick:** Return indices immediately upon finding the complement.

```
static int[] twoSum(int[] a, int target) {
    HashMap<Integer, Integer> map = new HashMap<>();
    for (int i = 0; i < a.length; i++) {
        int need = target - a[i];
        if (map.containsKey(need)) return new int[]{map.get(need), i};
        map.put(a[i], i);
    }
    return new int[]{-1, -1};
}
```

#7 First Unique Char

EASY FREQUENCY ARRAY *Frequency array is faster than sort.*

Time / Space	O(N) / O(1)	I/O Sample	In: "leetcode" → Out: 'l'

💡 **Interview Trick:** Fixed 256 array handles all ASCII without HashMap overhead.

```java
static char firstUnique(String s) {
    int[] f = new int[256];
    for (char c : s.toCharArray()) f[c]++;
    for (char c : s.toCharArray())
        if (f[c] == 1) return c;
    return ' ';
}
```

#8 Anagram Check

EASY FREQUENCY ARRAY *Frequency array matching.*

Time / Space	O(N) / O(1)	I/O Sample	In: "listen", "silent" → Out: true

💡 **Interview Trick:** Decrement frequency in second pass to find mismatches.

```java
static boolean isAnagram(String a, String b) {
    if (a.length() != b.length()) return false;
    int[] f = new int[256];
    for (char c : a.toCharArray()) f[c]++;
    for (char c : b.toCharArray())
        if (--f[c] < 0) return false;
    return true;
}
```

#9 Longest Common Prefix

EASY STRING/ITERATION *Shrink prefix until all match.*

Time / Space	O(N*M) / O(1)	I/O Sample	In: ["flower","flow","flight"] → Out: "fl"

💡 **Interview Trick:** StartsWith is highly optimized in Java.

```java
static String lcp(String[] arr) {
    if (arr.length == 0) return "";
    String p = arr[0];
    for (int i = 1; i < arr.length; i++) {
        while (!arr[i].startsWith(p))
            p = p.substring(0, p.length() - 1);
    }
    return p.isEmpty() ? "" : p;
}
```

#10 Maximum Subarray

MEDIUM KADANE'S ALGORITHM *Restart or extend at each step.*

Time / Space	O(N) / O(1)	I/O Sample	In: [-2,1,-3,4,-1,2,1,-5,4] → Out: 6

💡 **Interview Trick:** If current sum becomes negative, it's better to restart.

```java
static int maxSubarray(int[] a) {
    int best = a[0], cur = a[0];
    for (int i = 1; i < a.length; i++) {
        cur = Math.max(a[i], cur + a[i]);
        best = Math.max(best, cur);
    }
    return best;
}
```

#11 Merge Sorted Arrays

EASY TWO POINTERS *Take the smaller front element.*

Time / Space	O(N+M) / O(N+M)	I/O Sample	In: [1,3], [2,4] → Out: [1,2,3,4]

💡 **Interview Trick:** Both arrays must already be sorted! Can be done in-place from the back if first array has extra space.

```
static int[] merge(int[] a, int[] b) {
    int[] res = new int[a.length + b.length];
    int i = 0, j = 0, k = 0;
    while (i < a.length && j < b.length) res[k++] = a[i] < b[j] ? a[i++] : b[j++];
    while (i < a.length) res[k++] = a[i++];
    while (j < b.length) res[k++] = b[j++];
    return res;
}
```

#12 Rotate Array by K

MEDIUM MATH/REVERSAL *Reverse whole, then parts.*

Time / Space	O(N) / O(1)	I/O Sample	In: [1,2,3,4,5], k=2 → Out: [4,5,1,2,3]

💡 **Interview Trick:** Always do k %= n to prevent redundant rotations.

```
static void rotate(int[] a, int k) {
    if (a.length == 0) return;
    k %= a.length;
    rev(a, 0, a.length - 1);
    rev(a, 0, k - 1);
    rev(a, k, a.length - 1);
}
static void rev(int[] a, int l, int r) {
    while (l < r) { int t = a[l]; a[l] = a[r]; a[r] = t; l++; r--; }
}
```

#13 Binary Search

EASY BINARY SEARCH *Keep halving the search space.*

Time / Space	O(log N) / O(1)	I/O Sample	In: [1,2,3,4,5], x=4 → Out: 3

💡 **Interview Trick:** Use l + (r-l)/2 to prevent integer overflow.

```
static int bs(int[] a, int x) {
    int l = 0, r = a.length - 1;
    while (l <= r) {
        int m = l + (r - l) / 2;
        if (a[m] == x) return m;
        if (a[m] < x) l = m + 1;
        else r = m - 1;
    }
    return -1;
}
```

#14 Search Rotated Array

MEDIUM BINARY SEARCH *One half is always sorted.*

Time / Space	O(log N) / O(1)	I/O Sample	In: [4,5,1,2,3], target=1 → Out: 2

💡 **Interview Trick:** Determine which half is strictly sorted first.

```
static int search(int[] a, int target) {
    int l = 0, r = a.length - 1;
    while (l <= r) {
        int m = l + (r - l) / 2;
        if (a[m] == target) return m;
        if (a[l] <= a[m]) {
            if (a[l] <= target && target < a[m]) r = m - 1; else l = m + 1;
        } else {
            if (a[m] < target && target <= a[r]) l = m + 1; else r = m - 1;
        }
    }
    return -1;
}
```

#15 Valid Parentheses

EASY STACK *Push openers, match closers.*

Time / Space	O(N) / O(N)	I/O Sample	In: "()[]{}" → Out: true

💡 **Interview Trick:** Can push expected closing bracket to simplify matching logic.

```
static boolean valid(String s) {
    Stack<Character> st = new Stack<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '{' || c == '[') st.push(c);
        else {
            if (st.isEmpty()) return false;
            char t = st.pop();
            if ((c == ')' && t != '(') || (c == '}' && t != '{') || (c == ']' && t != '[')) retur
        }
    }
    return st.isEmpty();
}
```

#16 Next Greater Element

MEDIUM MONOTONIC STACK *Monotonic decreasing stack.*

Time / Space	O(N) / O(N)	I/O Sample	In: [2,1,2,4,3] → Out: [4,2,4,-1,-1]

💡 **Interview Trick:** Stack stores indices, not values, for positional distance.

```
static int[] nextGreater(int[] a) {
    int n = a.length;
    int[] res = new int[n];
    Arrays.fill(res, -1);
    Stack<Integer> st = new Stack<>();
    for (int i = 0; i < n; i++) {
        while (!st.isEmpty() && a[i] > a[st.peek()]) res[st.pop()] = a[i];
        st.push(i);
    }
    return res;
}
```

#17 Daily Temperatures

MEDIUM MONOTONIC STACK *Same stack, but store distance.*

Time / Space	O(N) / O(N)	I/O Sample	In: [73,74,75,71,69,72,76,73] → Out: [1,1,4,2,1,1,0,0]

💡 **Interview Trick:** Pop when finding a warmer day and record index difference.

```
static int[] temperatures(int[] a) {
    int n = a.length;
    int[] res = new int[n];
    Stack<Integer> st = new Stack<>();
    for (int i = 0; i < n; i++) {
        while (!st.isEmpty() && a[i] > a[st.peek()]) {
            int j = st.pop();
            res[j] = i - j;
        }
        st.push(i);
    }
    return res;
}
```

#18 Reverse Linked List

EASY LINKED LIST *Use prev, cur, next.*

Time / Space	O(N) / O(1)	I/O Sample	In: 1->2->3->null → Out: 3->2->1->null

💡 **Interview Trick:** Always temporarily store cur.next before mutating pointers.

```
static Node reverseList(Node head) {
    Node prev = null, cur = head;
    while (cur != null) {
        Node nxt = cur.next;
        cur.next = prev;
        prev = cur;
        cur = nxt;
    }
    return prev;
}
```

#19 Detect Cycle in LL

EASY FAST & SLOW POINTERS *Slow-fast pointers meet inside cycle.*

Time / Space	O(N) / O(1)	I/O Sample	In: 1->2->3->2... → Out: true

💡 **Interview Trick:** Floyd's Tortoise and Hare algorithm guarantees a meeting.

```
static boolean hasCycle(Node head) {
    Node slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}
```

#20 Merge Two Sorted LLs

EASY LINKED LIST *Use dummy node.*

Time / Space	O(N+M) / O(1)	I/O Sample	In: 1->3, 2->4 → Out: 1->2->3->4

💡 **Interview Trick:** Dummy node simplifies edge cases on the head pointer.

```
static Node mergeLists(Node a, Node b) {
    Node dummy = new Node(0), cur = dummy;
    while (a != null && b != null) {
        if (a.val < b.val) { cur.next = a; a = a.next; }
        else { cur.next = b; b = b.next; }
        cur = cur.next;
    }
    cur.next = (a != null) ? a : b;
    return dummy.next;
}
```

#21 Level Order Traversal

MEDIUM BFS *BFS uses a queue.*

Time / Space	O(N) / O(N)	I/O Sample	In: [3,9,20,null,null,15,7] → Out: 3 9 20 15 7

💡 **Interview Trick:** Poll, Print, then Offer Left and Right children.

```
static void levelOrder(TNode root) {
    if (root == null) return;
    Queue<TNode> q = new LinkedList<>();
    q.offer(root);
    while (!q.isEmpty()) {
        TNode cur = q.poll();
        System.out.print(cur.val + " ");
        if (cur.left != null) q.offer(cur.left);
        if (cur.right != null) q.offer(cur.right);
    }
}
```

#22 Height of Binary Tree

EASY DFS *1 + max(left, right).*

Time / Space	O(N) / O(H)	I/O Sample	In: [3,9,20,null,null,15,7] → Out: 3

💡 **Interview Trick:** Recursion handles base case of null node automatically.

```
static int height(TNode root) {
    if (root == null) return 0;
    return 1 + Math.max(height(root.left), height(root.right));
}
```

#23 Number of Islands

MEDIUM DFS/MATRIX *DFS each connected land component.*

Time / Space	O(M*N) / O(M*N)	I/O Sample	In: [['1','0'],['0','1']] → Out: 2

💡 **Interview Trick:** Mark visited by changing '1' to '0' or using boolean array.

```
static int islands(char[][] g) {
    int m = g.length, n = g[0].length, count = 0;
    boolean[][] vis = new boolean[m][n];
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            if (g[i][j] == '1' && !vis[i][j]) { dfs(g, vis, i, j); count++; }
    return count;
}
static void dfs(char[][] g, boolean[][] vis, int i, int j) {
    if (i < 0 || j < 0 || i >= g.length || j >= g[0].length || g[i][j] == '0' || vis[i][j]) retur
    vis[i][j] = true;
    dfs(g, vis, i + 1, j); dfs(g, vis, i - 1, j); dfs(g, vis, i, j + 1); dfs(g, vis, i, j - 1);
}
```

#24 Flood Fill

EASY DFS/MATRIX *Repaint same-colored region.*

Time / Space	O(M*N) / O(M*N)	I/O Sample	In: [[1,1],[1,0]], color=2 → Out: [[2,2],[2,0]]

💡 **Interview Trick:** Prevent infinite loop by checking if old color == new color.

```
static void fill(int[][] img, int sr, int sc, int color) {
    int old = img[sr][sc];
    if (old == color) return;
    ff(img, sr, sc, old, color);
}
static void ff(int[][] img, int i, int j, int old, int color) {
    if (i < 0 || j < 0 || i >= img.length || j >= img[0].length || img[i][j] != old) return;
    img[i][j] = color;
    ff(img, i + 1, j, old, color); ff(img, i - 1, j, old, color);
    ff(img, i, j + 1, old, color); ff(img, i, j - 1, old, color);
}
```

#25 Topological Sort

MEDIUM KAHN'S ALGORITHM (BFS) *Indegree array + queue.*

Time / Space	O(V+E) / O(V)	I/O Sample	In: V=2, edges=[[1,0]] → Out: [0,1]

💡 **Interview Trick:** Only Directed Acyclic Graphs (DAGs) can be topologically sorted.

```java
static int[] topo(int V, int[][] edges) {
    List<List<Integer>> g = new ArrayList<>();
    for (int i = 0; i < V; i++) g.add(new ArrayList<>());
    int[] indeg = new int[V];
    for (int[] e : edges) { g.get(e[0]).add(e[1]); indeg[e[1]]++; }
    Queue<Integer> q = new LinkedList<>();
    for (int i = 0; i < V; i++) if (indeg[i] == 0) q.offer(i);
    int[] ans = new int[V]; int k = 0;
    while (!q.isEmpty()) {
        int u = q.poll(); ans[k++] = u;
        for (int v : g.get(u)) if (--indeg[v] == 0) q.offer(v);
    }
    return ans;
}
```

#26 Dijkstra Shortest Path

MEDIUM GREEDY/HEAP *Min-heap picks closest node.*

Time / Space	O(E log V) / O(V)	I/O Sample	In: V=3, src=0, edges=... → Out: dist array

💡 **Interview Trick:** Avoid relaxing paths that are already longer than known dist.

```java
static int[] dijkstra(List<List<int[]>> g, int src) {
    int n = g.size();
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[0]));
    pq.offer(new int[]{0, src});
    while (!pq.isEmpty()) {
        int[] cur = pq.poll();
        int d = cur[0], u = cur[1];
        if (d != dist[u]) continue;
        for (int[] e : g.get(u)) {
            int v = e[0], w = e[1];
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.offer(new int[]{dist[v], v});
            }
        }
    }
    return dist;
}
```

#27 Coin Change

MEDIUM DYNAMIC PROGRAMMING *DP on amount.*

Time / Space	O(N * amt) / O(amt)	I/O Sample	In: coins=[1,2,5], amt=11 → Out: 3

💡 **Interview Trick:** Initialize DP array with a large safe value, not Integer.MAX_VALUE to avoid overflow.

```java
static int coinChange(int[] c, int amt) {
    int INF = 1_000_000;
    int[] dp = new int[amt + 1];
    Arrays.fill(dp, INF);
    dp[0] = 0;
    for (int coin : c)
        for (int i = coin; i <= amt; i++)
            dp[i] = Math.min(dp[i], dp[i - coin] + 1);
    return dp[amt] >= INF ? -1 : dp[amt];
}
```

#28 House Robber

MEDIUM DYNAMIC PROGRAMMING *Pick or skip, never adjacent.*

Time / Space	O(N) / O(1)	I/O Sample	In: [1,2,3,1] → Out: 4

💡 **Interview Trick:** You only need the last two calculated values to proceed.

```java
static int rob(int[] a) {
    int prev = 0, cur = 0;
    for (int x : a) {
        int temp = cur;
        cur = Math.max(cur, prev + x);
        prev = temp;
    }
    return cur;
}
```

#29 Longest Unique Substr

MEDIUM SLIDING WINDOW *Sliding window + last seen index.*

Time / Space	O(N) / O(1)	I/O Sample	In: "abcabcbb" → Out: 3

💡 **Interview Trick:** Jump the left pointer to last_seen + 1 instantly.

```java
static int longestUnique(String s) {
    HashMap<Character, Integer> map = new HashMap<>();
    int l = 0, ans = 0;
    for (int r = 0; r < s.length(); r++) {
        char c = s.charAt(r);
        if (map.containsKey(c)) l = Math.max(l, map.get(c) + 1);
        map.put(c, r);
        ans = Math.max(ans, r - l + 1);
    }
    return ans;
}
```

#30 Min Swaps to Sort

HARD GRAPH/CYCLES *Sort positions and count cycles.*

Time / Space	O(N log N) / O(N)	I/O Sample	In: [4,3,2,1] → Out: 2

💡 **Interview Trick:** Number of swaps for a cycle of length L is L - 1.

```
static int minSwaps(int[] arr) {
    int n = arr.length;
    int[][] p = new int[n][2];
    for (int i = 0; i < n; i++) { p[i][0] = arr[i]; p[i][1] = i; }
    Arrays.sort(p, Comparator.comparingInt(a -> a[0]));
    boolean[] vis = new boolean[n];
    int ans = 0;
    for (int i = 0; i < n; i++) {
        if (vis[i] || p[i][1] == i) continue;
        int cycle = 0, j = i;
        while (!vis[j]) { vis[j] = true; j = p[j][1]; cycle++; }
        if (cycle > 1) ans += cycle - 1;
    }
    return ans;
}
```

#31 Second Largest Element

EASY LINEAR SCAN *Track first and second in one pass.*

Time / Space	O(N) / O(1)	I/O Sample	In: [10, 5, 10] → Out: 5

💡 **Interview Trick:** Handle duplicates by ensuring second is strictly less than first.

```
static int secondLargest(int[] a) {
    int first = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
    for (int x : a) {
        if (x > first) { second = first; first = x; }
        else if (x > second && x < first) second = x;
    }
    return second;
}
```

#32 Move Zeroes

EASY TWO POINTERS *Use one write pointer.*

Time / Space	O(N) / O(1)	I/O Sample	In: [0,1,0,3,12] → Out: [1,3,12,0,0]

💡 **Interview Trick:** Swap non-zeros to the front, zeroes naturally pushed back.

```
static void moveZeroes(int[] a) {
    int j = 0;
    for (int i = 0; i < a.length; i++) {
        if (a[i] != 0) {
            int t = a[i]; a[i] = a[j]; a[j] = t;
            j++;
        }
    }
}
```

#33 Missing Number

EASY MATH/BITWISE *Expected sum minus actual sum.*

Time / Space	O(N) / O(1)	I/O Sample	In: [3,0,1] → Out: 2

💡 **Interview Trick:** Alternative: XOR also works and avoids integer overflow.

```
static int missingNumber(int[] a) {
    int n = a.length, sum = n * (n + 1) / 2, s = 0;
    for (int x : a) s += x;
    return sum - s;
}
```

#34 Majority Element

EASY BOYER-MOORE *Boyer-Moore cancels pairs.*

Time / Space	O(N) / O(1)	I/O Sample	In: [2,2,1,1,1,2,2] → Out: 2

💡 **Interview Trick:** Only works if a majority element strictly > N/2 exists.

```
static int majorityElement(int[] a) {
    int cand = 0, count = 0;
    for (int x : a) {
        if (count == 0) { cand = x; count = 1; }
        else if (x == cand) count++;
        else count--;
    }
    return cand;
}
```

#35 Best Time to Buy/Sell

EASY GREEDY *Keep the lowest buy price.*

Time / Space	O(N) / O(1)	I/O Sample	In: [7,1,5,3,6,4] → Out: 5

💡 **Interview Trick:** Profit is simply max(current_price - min_so_far).

```
static int maxProfit(int[] a) {
    int min = a[0], profit = 0;
    for (int x : a) {
        min = Math.min(min, x);
        profit = Math.max(profit, x - min);
    }
    return profit;
}
```

#36 Contains Duplicate

EASY HASHING *If add fails, duplicate found.*

Time / Space	O(N) / O(N)	I/O Sample	In: [1,2,3,1] → Out: true

💡 **Interview Trick:** HashSet.add() returns false if element already exists.

```
static boolean hasDuplicate(int[] a) {
    HashSet<Integer> set = new HashSet<>();
    for (int x : a) if (!set.add(x)) return true;
    return false;
}
```

#37 Intersection of Arrays

EASY HASHING *Use HashSet for fast lookup.*

Time / Space	O(N+M) / O(min(N,M))	I/O Sample	In: [1,2,2,1], [2,2] → Out: [2]

💡 **Interview Trick:** Remove from set after finding match to avoid duplicates.

```
static int[] intersection(int[] a, int[] b) {
    HashSet<Integer> set = new HashSet<>();
    for (int x : a) set.add(x);
    ArrayList<Integer> res = new ArrayList<>();
    for (int x : b) if (set.contains(x)) { res.add(x); set.remove(x); }
    return res.stream().mapToInt(i -> i).toArray();
}
```

#38 Count Frequency

EASY HASHING *Use getOrDefault().*

Time / Space	O(N) / O(N)	I/O Sample	In: [1,1,2] → Out: {1=2, 2=1}

💡 **Interview Trick:** Simplifies logic over checking map.containsKey().

```
static void freq(int[] a) {
    HashMap<Integer, Integer> map = new HashMap<>();
    for (int x : a) map.put(x, map.getOrDefault(x, 0) + 1);
    System.out.println(map);
}
```

#39 LCS

MEDIUM DYNAMIC PROGRAMMING *Match -> diag; else max(left, up).*

Time / Space	O(N*M) / O(N*M)	I/O Sample	In: "abcde", "ace" → Out: 3

💡 **Interview Trick:** Can be optimized to O(M) space using two rows.

```
static int lcs(String a, String b) {
    int n = a.length(), m = b.length();
    int[][] dp = new int[n + 1][m + 1];
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            dp[i][j] = a.charAt(i - 1) == b.charAt(j - 1) ? 1 + dp[i - 1][j - 1] : Math.max(dp[i
    return dp[n][m];
}
```

#40 Climbing Stairs

EASY DP/FIBONACCI *Same as Fibonacci sequence.*

Time / Space	O(N) / O(1)	I/O Sample	In: n=3 → Out: 3

💡 **Interview Trick:** Number of ways to reach step N is step N-1 + step N-2.

```
static int climbStairs(int n) {
    if (n <= 2) return n;
    int a = 1, b = 2;
    for (int i = 3; i <= n; i++) {
        int c = a + b;
        a = b; b = c;
    }
    return b;
}
```

#41 Power of Two

EASY BIT MANIPULATION *Only one set bit.*

Time / Space	O(1) / O(1)	I/O Sample	In: 16 → Out: true

💡 **Interview Trick:** n & (n-1) removes the lowest set bit.

```
static boolean isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}
```

#42 Single Number

EASY BIT MANIPULATION *XOR cancels duplicates.*

Time / Space	O(N) / O(1)	I/O Sample	In: [4,1,2,1,2] → Out: 4

💡 **Interview Trick:** A XOR A = 0; A XOR 0 = A.

```
static int singleNumber(int[] a) {
    int x = 0;
    for (int v : a) x ^= v;
    return x;
}
```

#43 Factorial

EASY MATH *Multiply from 2 to n.*

Time / Space	O(N) / O(1)	I/O Sample	In: 5 → Out: 120

💡 **Interview Trick:** Use long to avoid integer overflow on inputs > 12.

```
static long fact(int n) {
    long ans = 1;
    for (int i = 2; i <= n; i++) ans *= i;
    return ans;
}
```

#44 Fibonacci

EASY MATH/DP *Keep only the last two values.*

Time / Space	O(N) / O(1)	I/O Sample	In: 5 → Out: 5

💡 **Interview Trick:** O(1) space is better than recursive O(N) stack space.

```java
static int fib(int n) {
    if (n <= 1) return n;
    int a = 0, b = 1;
    for (int i = 2; i <= n; i++) {
        int c = a + b;
        a = b; b = c;
    }
    return b;
}
```

#45 Check Prime

EASY MATH *Check divisors up to sqrt(N).*

Time / Space	O(sqrt(N)) / O(1)	I/O Sample	In: 17 → Out: true

💡 **Interview Trick:** Checking beyond square root is redundant.

```java
static boolean isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; i * i <= n; i++)
        if (n % i == 0) return false;
    return true;
}
```

#46 Reverse String

EASY STRING *StringBuilder.reverse().*

Time / Space	O(N) / O(N)	I/O Sample	In: "hello" → Out: "olleh"

💡 **Interview Trick:** In Java, Strings are immutable, so SB is necessary.

```java
static String reverse(String s) {
    return new StringBuilder(s).reverse().toString();
}
```

#47 Integer Palindrome

EASY MATH *Reverse digits and compare.*

Time / Space	O(log N) / O(1)	I/O Sample	In: 121 → Out: true

💡 **Interview Trick:** May overflow for very large integers. Avoid String conversion to keep space complexity O(1).

```java
static boolean isPalindrome(int n) {
    int x = n, rev = 0;
    while (x > 0) {
        rev = rev * 10 + x % 10;
        x /= 10;
    }
    return rev == n;
}
```

#48 Sum of Array

EASY ARRAY *Accumulate in one pass.*

Time / Space	O(N) / O(1)	I/O Sample	In: [1,2,3] → Out: 6

💡 **Interview Trick:** Use standard enhanced for-loop for clean syntax.

```java
static int sum(int[] a) {
    int s = 0;
    for (int x : a) s += x;
    return s;
}
```

#49 Find Duplicates

MEDIUM HASHING/ARRAY *Frequency count identifies repeats.*

Time / Space	O(N) / O(N)	I/O Sample	In: [4,3,2,7,8,2,3,1] → Out: 2, 3

💡 **Interview Trick:** Can be O(1) space by marking array elements negative if elements in [1, N].

```java
static void duplicates(int[] a) {
    HashMap<Integer, Integer> map = new HashMap<>();
    for (int x : a) map.put(x, map.getOrDefault(x, 0) + 1);
    for (var e : map.entrySet())
        if (e.getValue() > 1) System.out.println(e.getKey());
}
```

#50 Max Element

EASY LINEAR SCAN *Update max while scanning once.*

Time / Space	O(N) / O(1)	I/O Sample	In: [1,5,3,9,2] → Out: 9

💡 **Interview Trick:** Initialize with a[0], not 0, to handle negatives.

```
static int max(int[] a) {
    int m = a[0];
    for (int x : a) if (x > m) m = x;
    return m;
}
```

#51 Character Frequency

EASY ARRAY HASHING *Use an int[256] array.*

Time / Space	O(N) / O(1)	I/O Sample	In: "hello" → Out: [... h=1, e=1, l=2, o=1 ...]

💡 **Interview Trick:** Array indexing is significantly faster than HashMap wrapping.

```
static void charFreq(String s) {
    int[] f = new int[256];
    for (char c : s.toCharArray()) f[c]++;
    for (int i = 0; i < 256; i++)
        if (f[i] > 0) System.out.println((char)i + " : " + f[i]);
}
```

#52 Merge Intervals

MEDIUM SORTING/INTERVALS *Sort by start, then merge.*

Time / Space	O(N log N) / O(N)	I/O Sample	In: [[1,3],[2,6]] → Out: [[1,6]]

💡 **Interview Trick:** Always verify array is not empty before checking last index.

```java
static int[][] merge(int[][] a) {
    java.util.Arrays.sort(a, (x, y) -> x[0] - y[0]);
    java.util.ArrayList<int[]> res = new java.util.ArrayList<>();
    for (int[] in : a) {
        if (res.isEmpty() || res.get(res.size() - 1)[1] < in[0]) res.add(in);
        else res.get(res.size() - 1)[1] = Math.max(res.get(res.size() - 1)[1], in[1]);
    }
    return res.toArray(new int[0][]);
}
```

#53 Prefix Sum

EASY PREFIX ARRAY *Build running sum once.*

Time / Space	O(N) / O(N)	I/O Sample	In: arr=[1,2,3], query=[0,2] → Out: 6

💡 **Interview Trick:** Pre-allocating array size + 1 simplifies boundary conditions.

```java
static int rangeSum(int[] a, int l, int r) {
    int[] pre = new int[a.length + 1];
    for (int i = 0; i < a.length; i++) pre[i + 1] = pre[i] + a[i];
    return pre[r + 1] - pre[l];
}
```

#54 Queue Basics

EASY QUEUE *offer() adds, poll() removes.*

Time / Space	- / -	I/O Sample	In: offer(10), offer(20) → Out: 10

💡 **Interview Trick:** Use LinkedList or ArrayDeque as implementation for Queue.

```
static void queueDemo() {
    java.util.Queue<Integer> q = new java.util.LinkedList<>();
    q.offer(10);
    q.offer(20);
    System.out.println(q.poll());
}
```

#55 Group Anagrams

MEDIUM HASHMAP/SORTING *Group values by sorted key pattern.*

Time / Space	O(N * K log K) / O(N * K)	I/O Sample	In: ["eat","tea","tan"] → Out: [["eat","tea"],["tan"]]

💡 **Interview Trick:** Key = Sorted String. computeIfAbsent elegantly handles list initialization.

```
static java.util.List<java.util.List<String>> groupAnagrams(String[] arr) {
    java.util.HashMap<String, java.util.List<String>> map = new java.util.HashMap<>();
    for (String s : arr) {
        char[] ch = s.toCharArray();
        java.util.Arrays.sort(ch);
        String key = new String(ch);
        map.computeIfAbsent(key, k -> new java.util.ArrayList<>()).add(s);
    }
    return new java.util.ArrayList<>(map.values());
}
```

#56 Merge Intervals

MEDIUM SORTING / INTERVALS *Sort by start, then merge overlaps.*

Time / Space	O(N log N) / O(N)	I/O Sample	In: [[1,3],[2,6],[8,10]] → Out: [[1,6],[8,10]]

💡 **Interview Trick:** Always sort intervals by start time first to ensure linear merge scan.

```java
static int[][] merge(int[][] a) {
    java.util.Arrays.sort(a, (x, y) -> x[0] - y[0]);
    java.util.ArrayList<int[]> res = new java.util.ArrayList<>();
    for (int[] in : a) {
        if (res.isEmpty() || res.get(res.size() - 1)[1] < in[0])
            res.add(in);
        else
            res.get(res.size() - 1)[1] = Math.max(res.get(res.size() - 1)[1], in[1]);
    }
    return res.toArray(new int[0][]);
}
```

#57 Kadane's Algorithm

MEDIUM DYNAMIC PROGRAMMING / GREEDY *Restart or extend at each step.*

Time / Space	O(N) / O(1)	I/O Sample	In: [-2, 1, -3, 4, -1, 2, 1] → Out: 6

💡 **Interview Trick:** Keeps track of maximum subarray sum ending at current index.

```java
static int maxSubarray(int[] a) {
    int best = a[0], cur = a[0];
    for (int i = 1; i < a.length; i++) {
        cur = Math.max(a[i], cur + a[i]);
        best = Math.max(best, cur);
    }
    return best;
}
```

#58 Sliding Window

MEDIUM SLIDING WINDOW / HASHING *Expand right, shrink left.*

Time / Space	O(N) / O(min(N, M))	I/O Sample	In: "abcabcbb" → Out: 3

💡 **Interview Trick:** Storing char index in HashMap allows left pointer to jump directly.

```java
static int longestUnique(String s) {
    java.util.HashMap<Character, Integer> map = new java.util.HashMap<>();
    int left = 0, ans = 0;
    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        if (map.containsKey(c)) left = Math.max(left, map.get(c) + 1);
        map.put(c, right);
        ans = Math.max(ans, right - left + 1);
    }
    return ans;
}
```

#59 Binary Search

EASY BINARY SEARCH *Keep halving the search space.*

Time / Space	O(log N) / O(1)	I/O Sample	In: [1, 3, 5, 7, 9], target=5 → Out: 2

💡 **Interview Trick:** Calculate mid using left + (right - left) / 2 to prevent integer overflow.

```java
static int bs(int[] a, int x) {
    int left = 0, right = a.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (a[mid] == x) return mid;
        if (a[mid] < x) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

#60 Prefix Sum

EASY PREFIX ARRAY *Build running sum once.*

Time / Space	O(N) / O(N)	I/O Sample	In: a=[1,2,3,4], l=1, r=3 → Out: 9

💡 **Interview Trick:** pre[r + 1] - pre[l] gives sum of elements from index l to r.

```
static int rangeSum(int[] a, int l, int r) {
    int[] pre = new int[a.length + 1];
    for (int i = 0; i < a.length; i++) pre[i + 1] = pre[i] + a[i];
    return pre[r + 1] - pre[l];
}
```

#61 Next Greater Element

MEDIUM MONOTONIC STACK *Use a decreasing stack.*

Time / Space	O(N) / O(N)	I/O Sample	In: [4, 5, 2, 25] → Out: [5, 25, 25, -1]

💡 **Interview Trick:** Pop elements from stack as soon as a greater value is encountered.

```
static int[] nextGreater(int[] a) {
    int n = a.length;
    int[] res = new int[n];
    java.util.Arrays.fill(res, -1);
    java.util.Stack<Integer> st = new java.util.Stack<>();
    for (int i = 0; i < n; i++) {
        while (!st.isEmpty() && a[i] > a[st.peek()]) res[st.pop()] = a[i];
        st.push(i);
    }
    return res;
}
```

#62 Stack Basics

EASY STACK *push adds, pop removes and returns.*

Time / Space	O(N) / O(N)	I/O Sample	In: "({[]})" → Out: true

💡 **Interview Trick:** An empty stack at the end indicates all opening brackets were balanced.

```
static boolean valid(String s) {
    java.util.Stack<Character> st = new java.util.Stack<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '{' || c == '[') st.push(c);
        else {
            if (st.isEmpty()) return false;
            char t = st.pop();
            if ((c == ')' && t != '(') || (c == '}' && t != '{') || (c == ']' && t != '[')) retur
        }
    }
    return st.isEmpty();
}
```

#63 Queue Basics

EASY QUEUE (FIFO) *offer adds, poll removes and returns.*

Time / Space	- / -	I/O Sample	In: offer(10), offer(20) → Out: 10

💡 **Interview Trick:** Use ArrayDeque or LinkedList as standard Java Queue implementations.

```
static void queueDemo() {
    java.util.Queue<Integer> q = new java.util.LinkedList<>();
    q.offer(10);
    q.offer(20);
    System.out.println(q.poll());
}
```

#64 HashMap Tricks

MEDIUM | HASHING / STRING MANIPULATION | *Group values by key pattern.*

Time / Space	O(N * K log K) / O(N * K)	I/O Sample	In: ["eat","tea","tan"] → Out: [["eat","tea"],["tan"]]

💡 **Interview Trick:** Key = Sorted String. computeIfAbsent simplifies list creation inside HashMap lookup.

```
static java.util.List<java.util.List<String>> groupAnagrams(String[] arr) {
    java.util.HashMap<String, java.util.List<String>> map = new java.util.HashMap<>();
    for (String s : arr) {
        char[] ch = s.toCharArray();
        java.util.Arrays.sort(ch);
        String key = new String(ch);
        map.computeIfAbsent(key, k -> new java.util.ArrayList<>()).add(s);
    }
    return new java.util.ArrayList<>(map.values());
}
```

#65 StringBuilder Tricks

EASY | STRING MANIPULATION | *Use StringBuilder for fast modify operations.*

Time / Space	O(N) / O(N)	I/O Sample	In: "infosys" → Out: "sysofni"

💡 **Interview Trick:** StringBuilder.reverse() mutates internal char array buffer in-place.

```
static String reverse(String s) {
    return new StringBuilder(s).reverse().toString();
}
```

#66 Bit Manipulation Basics

EASY BIT MANIPULATION (XOR) *XOR cancels duplicates.*

Time / Space	O(N) / O(1)	I/O Sample	In: [4, 1, 2, 1, 2] → Out: 4

💡 **Interview Trick:** XOR is associative and commutative; all paired numbers cancel out completely.

```
static int singleNumber(int[] a) {
    int x = 0;
    for (int v : a) x ^= v;
    return x;
}
```